

Data Structures

Mid Term Examination Solutions (E1PY203B)

Section A: Fundamentals & Arrays

1. What is a Sparse Matrix? Explain its representation.

A Sparse Matrix is a matrix in which most of the elements are zero. Storing all these zeros is inefficient in terms of memory. **Representation:** Instead of a 2D array, it is represented using a 3-tuple format (Triplet Representation) where each non-zero element is stored as: `(row, column, value)`.

2. Explain the concept of Time-Space Trade-off with a suitable example.

Time-Space Trade-off is a situation where you can reduce the execution time of an algorithm at the cost of using more memory, or vice versa. **Example:** In a photo viewer or browser, caching recently viewed items (using memory) allows for near-instant retrieval (saving time) when the user goes back.

3. Explain Single Dimensional and Multidimensional Arrays with examples.

- **Single Dimensional Array:** A linear list of elements of the same type stored in contiguous memory. Example: `int marks[5] = {90, 85, 70, 95, 80};`
- **Multidimensional Array:** An array of arrays, representing data in a grid or table format. Example: A 2D array `int matrix[2][3];` stores data in 2 rows and 3 columns.

Section B: Recursion & Queues

4. Write a recursive function for Tower of Hanoi and explain the number of moves for n=3.

Function: `void TOH(int n, char beg, char aux, char end)`. If $n=1$, move disk from beg to end. Else:

1. `TOH(n-1, beg, end, aux)`
2. Move disk from beg to end
3. `TOH(n-1, aux, beg, end)`

For $n=3$: The total number of moves is $2^n - 1 = 2^3 - 1 = 7$ moves.

5. What is a Deque and explain its types?

A Deque (Double-Ended Queue) is a linear data structure where insertion and deletion can be performed from both ends (front and rear).

- **Input Restricted Deque:** Deletion from both ends, but insertion only at one end.
- **Output Restricted Deque:** Insertion at both ends, but deletion only from one end.

Section C: Linked Lists & Applications

6. Describe the structure and working of a Doubly Linked List (DLL).

A Doubly Linked List contains nodes that have three parts: a **Data** field, a **Next** pointer (to the succeeding node), and a **Prev** pointer (to the preceding node).

Advantages: It allows for bidirectional traversal (forward and backward) and easier deletion of a node if the pointer to it is known.

7. Design a solution for a photo viewer application using a Doubly Linked List.

A photo viewer needs to navigate 'Forward' and 'Backward'. A DLL is ideal:

- Each node stores an image object or its file path.
- **Next** pointer links to the next image in the gallery.
- **Prev** pointer links to the previous image.
- When the user clicks "Next", the app moves to `current -> next` ; for "Previous", it moves to `current -> prev` .